



Universidad de
Oviedo

Programación funcional

Adoptando el paradigma funcional en Java

Juan Luis Mateo Cerdán

```
public class Example {  
  
    private static int square(int number) {  
        return number * number;  
    }  
  
    public static void main(String[] args) {  
        List<Integer> numbers =  
            Arrays.asList(1, 2, 3, 4, 5, 6, 7);  
  
        List<Integer> square = numbers.stream()  
            .filter(number -> number % 2 != 0)  
            .map(Example::square).toList();  
  
        square.forEach(System.out::println);  
    }  
}
```

Índice

Introducción

Características funcionales en Java

Streams API

Principios fundamentales de la PF en Java

Conceptos adicionales

Resumen

Lecturas adicionales

¿Qué es la programación funcional?

- ▶ Un paradigma declarativo donde los programas se construyen como una composición de funciones
- ▶ Principios fundamentales:
 - ▶ Funciones puras
 - ▶ Estado no mutable
 - ▶ Transparencia referencial
 - ▶ Funciones de primera clase
 - ▶ Funciones de orden superior

¿Por qué programación funcional en Java?

- ▶ Java es principalmente OO, pero desde Java 8 hay soporte para características funcionales
- ▶ Beneficios:
 - ▶ Código más conciso y legible
 - ▶ Facilidad para el paralelismo
 - ▶ Menos efectos secundarios
 - ▶ Mejor soporte para la gestión de flujos

¿Por qué programación funcional en Java?

// Imperative style

```
List<Integer> listNum = List.of(1, 2, 3, 4, 5, 6, 7, 8);
int result = 0;
for (int i = 0; i < listNum.size(); i++) {
    if (listNum.get(i) % 2 == 0) {
        result += listNum.get(i) * 10;
    }
}
```

// Functional style

```
List<Integer> listNum = List.of(1, 2, 3, 4, 5, 6, 7, 8);
Integer result = listNum.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * 10)
    .reduce(0, (n, m) -> n + m);
```

Principales características funcionales en Java

- ▶ Expresiones lambda
- ▶ Interfaces funcionales
- ▶ Referencias a métodos
- ▶ Composición
- ▶ El tipo `Optional`
- ▶ API Streams

Expresiones lambda I

- ▶ Basadas en el **cálculo lambda**, desarrollado en 1930s por Alonzo Church
- ▶ Son, básicamente, funciones anónimas
- ▶ ¿Para que las necesitamos?
 - ▶ Definir una función directamente como argumento
 - ▶ Definir una función como resultado de otra función
 - ▶ Definir una función local dentro de otra función

Expresiones lambda II

► Sintaxis en Java

```
(par1 [, par2 ...]) -> expression  
par -> expression
```

► Ejemplos:

```
(a, b) -> a + b // adds a and b, and returns the result  
n -> n > 10 // returns true if n is greater than 10, false  
otherwise
```


Expresiones lambda III

► Beneficios frente a las clases anónimas

```
// Comparator with anonymous class
Collections.sort(list, new Comparator<String>() {
    @Override
    public int compare(String s1, String s2) {
        return s1.length() - s2.length();
    }
});

// Comparator with lambda
Collections.sort(list, (s1, s2) -> s1.length() - s2.length());
```

Interfaces funcionales

- ▶ Interfaces con un único método abstracto (SAM)
- ▶ Más habituales en `java.util.function`:
 - ▶ `Predicate<T>`
 - ▶ `Function<T, R>`
 - ▶ `Consumer<T>`
 - ▶ `Supplier<T>`
- ▶ Interfaces funcionales personalizadas:

```
@FunctionalInterface
interface Calculator {
    int add(int a, int b);
}
```

Interfaces funcionales II

```
// Predicate: test a condition
```

```
Predicate<String> isLong = s -> s.length() > 5;
```

```
// Function: transform input
```

```
Function<String, Integer> strLen = s -> s.length();
```

```
// Consumer: perform an action
```

```
Consumer<String> printer = s -> System.out.println(s);
```

```
// Supplier: provide a value
```

```
Supplier<Double> random = () -> Math.random();
```

Referencias a métodos

- ▶ Reemplazan a las funciones lambda al llamar a un método existente
- ▶ Sintaxis: `ClassName::methodName`
- ▶ Ejemplo:

```
List<String> names = List.of("Alice", "Bob", "John");  
  
names.forEach(System.out::println);  
// Equivalent: names.forEach(n -> System.out.println(n));
```

Composición funcional I

- ▶ Encadenar una o más funciones para obtener otras más versátiles
- ▶ La interfaz `Function` tiene los métodos `andThen()` y `compose()` para ello
- ▶ Ejemplo:

```
Function<Integer, Integer> f = x -> x + 1;  
Function<Integer, Integer> g = x -> x * 2;
```

```
Function<Integer, Integer> h1 = f.andThen(g); // g(f(x))  
Function<Integer, Integer> h2 = f.compose(g); // f(g(x))
```

```
System.out.println(h1.apply(1)); // Prints 4  
System.out.println(h2.apply(1)); // Prints 3
```

Composición funcional II

- ▶ Varias interfases funcionales tienen métodos para la composición
- ▶ La interfaz Comparator

```
Comparator<Car> c = Comparator.comparing(Car::getPrice)
    .reversed()
    .thenComparing(Car::getHP)
```

- ▶ La itnerfaz Predicate

```
Predicate<Car> whiteCar = (Car c) -> c.getColor().equals("
    white");
Predicate<Car> notWhiteCheapCar = whiteCar
    .negate()
    .and(c -> c.getPrice() < 20000);
```

La clase Optional

- ▶ Esta clase se incluyó para evitar lidiar con valores `null` (y `NullPointerException`)
- ▶ Se trata de un envoltorio que permite almacenar el valor en sí o nada
- ▶ Podemos comprobar si el valor está presente con `isPresent()` y obtenerlo con `get()`

```
if (opt.isPresent())  
    return opt.get();
```

- ▶ También podemos tener un valor por defecto en caso de que no lo haya

```
opt.orElse("Default value"); //returns the default value if  
    not value present
```

Fundamentos Streams API

- ▶ Los streams (flujos) nos permiten manejar colecciones de forma declarativa, mediante la composición de operaciones, y paralelización incorporada
- ▶ Algunas operaciones de los streams: map, filter, reduce, collect
- ▶ Tarea: de una lista de nombres saca aquellos que empiecen por J, ordenados y en mayúsculas

Con colecciones

```
List<String> names = List.of("Alice", "Bob", "John", "Jane");

List<String> filteredNames = new ArrayList<String>();
for(String s: names)
    if(s.startsWith("J"))
        filteredNames.add(s);

Collections.sort(filteredNames);

List<String> result = new ArrayList<String>();
for(String s: filteredNames)
    result.add(s.toUpperCase());

System.out.println(result); // [JANE, JOHN]
```

Con streams

```
List<String> names = List.of("Alice", "Bob", "John", "Jane");

List<String> result = names.stream()
    .filter(n -> n.startsWith("J"))
    .sorted()
    .map(String::toUpperCase)
    .collect(Collectors.toList());

System.out.println(result); // [JANE, JOHN]
```

Generar streams

- ▶ Como hemos visto en el ejemplo anterior, se puede generar un stream a partir de una colección usando el método `stream()`
- ▶ Desde un conjunto de valores con el método `Stream.of()`

```
Stream<Integer> stream = Stream.of(1,2,3,4,5);
```

- ▶ Desde un array con `Arrays.stream()`

```
int[] numbers = {1,2,3,4,5};  
Arrays.stream(numbers).forEach(System.out::println);
```

- ▶ “Infinitos” streams con los métodos `Stream.iterate()` o `Stream.generate()`

```
stream = Stream.iterate(1, n -> n +1);  
stream = Stream.generate(new Random()::nextInt);
```

Operaciones con streams

- ▶ Hay 2 tipos de operaciones:
 - ▶ Intermedias: devuelve un stream, para encadenar operaciones
 - ▶ Terminales: devuelve otra cosa, incluso **void**
- ▶ Las operaciones intermedias son **perezosas**, sólo se procesan cuando la operación terminal se invoca
- ▶ Esto permite introducir optimizaciones cuando se usan los streams

Filtrar y trocear

- ▶ El método `filter()` toma como argumento un predicado y produce un stream con los mismos elementos que validan ese predicado
- ▶ También podemos descartar elementos con las operaciones:
 - ▶ `distinct()` para mantener sólo elementos únicos
 - ▶ `limit()` para recuperar sólo los primeros elementos del stream
 - ▶ `skip()` para descartar algunos elementos del comienzo del stream

```
names.stream()  
    .filter(n -> n.contains(" "))  
    .distinct()  
    .limit(3);
```

Mapear I

- ▶ El concepto de mapear significa aplicar una función a cada elemento del stream
- ▶ El método `map()` devuelve un nuevo stream con los elementos que la función devuelve

```
List<Integer> namesLengths = names.stream()  
    .map(String::length).collect(Collectors.toList());
```

Mapear II

- ¿Qué ocurre si el stream contiene una estructura de datos anidada, como una lista de listas?

```
List<List<Integer>> list = Arrays.asList(Arrays.asList(1, 5, 9), Arrays.asList(6, 8, 11));
```

- El método `map()` aplicará la función a cada lista interna
- ¿Y si queremos el cuadrado del número en ambas listas?
- El método `flatMap()` aplana el resultado mapeado

```
list.stream().flatMap(Collection::stream).map(n -> n * n)
```

Búsqueda I

- ▶ Siempre es útil identificar algunos elementos para determinar si cumplen una condición
- ▶ La interfaz `Stream` incluye los métodos `findFirst()` y `findAny()`
- ▶ Devuelven un `Optional` porque el elemento puede no encontrarse

```
Random rnd = new Random(0);  
Stream<Integer> nums = Stream.generate(() -> rnd.nextInt(10));
```

```
Optional<Integer> number1st = nums.filter(n -> n > 5)  
    .findFirst();
```

```
Optional<Integer> numberAny = nums.filter(n -> n > 5)  
    .findAny();
```


Búsqueda II

- Los métodos `anyMatch()`, `allMatch()` y `noneMatch()` toman un predicado como argumento y devuelven un booleano

```
boolean b1 = nums.anyMatch(n -> n > 5);
```

```
boolean b2 = nums.allMatch(n -> n < 0);
```

```
boolean b3 = nums.noneMatch(n -> n >= 0);
```

- Estos métodos hacen uso del **corto-circuito**

Reducción

- ▶ Reducción significa combinar todos los elementos del stream para producir un solo elemento
- ▶ Por ejemplo, la suma de todos los números

```
int sum = nums.limit(10).reduce(0, (a, b) -> a + b);
```

- ▶ El método `reduce()` toma dos argumentos
 - ▶ El primer argumento es el valor inicial
 - ▶ El segundo es una función binaria para combinar el valor actual con el siguiente del stream

Principios fundamentales de la PF en Java

- ▶ Volvamos a los principios de la PF en Java
- ▶ Estos son:
 - ▶ Funciones puras
 - ▶ Inmutabilidad
 - ▶ Transparencia referencial
 - ▶ Funciones de orden superior
 - ▶ Funciones de primera clase

Funciones puras I

- ▶ Siempre devuelven el mismo valor por los mismos argumentos y no tienen efectos secundarios
 - ▶ No dependen de valores externos que puedan cambiar
 - ▶ No modifican valores externos

```
// Pure function
int square(int x) {
    return x * x;
}
```

```
// Impure function
// fails rule 1
int f() {
    return 2 * x;
}
```

```
// Impure function
// fails rule 2
void g() {
    x++;
}
```

Funciones puras II

- ▶ Incluso si usamos las extensiones funcionales de Java, debemos tener cuidado de no introducir efectos secundarios

```
List<String> results = new ArrayList<>();
```

```
names.stream()  
    .map(String::toUpperCase)  
    .forEach(results::add); // Avoid: better to use collect()
```

Inmutabilidad I

- ▶ Los lenguajes puramente funcionales no admiten tipos de datos mutables
- ▶ Los valores que pueden cambiar hacen el código más propenso a errores, difícil de entender y paralelizar
- ▶ La inmutabilidad facilita la creación de funciones puras
- ▶ Java proporciona colecciones inmutables y la palabra clave **final**
- ▶ Se recomienda usar datos inmutables (en la medida de lo posible)

Inmutabilidad II

```
public class Person {  
    private final String name;  
  
    public Person(String name){  
        this.name = name;  
    }  
  
    public getName(){  
        return name;  
    }  
}
```

```
public static void main(String[] args) {  
    final List<Person> as = List.of(new  
        Person("John"));  
}
```

Transparencia referencial

- ▶ Las expresiones son intercambiables con su valor
- ▶ Para ello es fundamental que no tengan efectos secundarios
- ▶ Esto implica que el compilador y/o la JVM pueden realizar optimizaciones, como el cacheo
- ▶ También ayuda a paralelizar la evaluación de expresiones

Funciones de orden superior

- ▶ Una función es de orden superior si:
 - ▶ toma al menos una función como argumento, como las funciones de `Stream map()` o `filter()`
 - ▶ devuelve una función, como la función `compose()` o `andThen()` de la interfaz `Function`
- ▶ Ejemplos personalizados:

```
// A function that checks if a number validates a given predicate
public boolean checkNumberFor(double n, Predicate<Double> pred) {
    return pred.test(n);
}
```

```
// A generic function to add a number (N) to other
Function<Integer, Function<Integer, Integer>> addN = n -> x -> x
    + n;
// Specific case for adding 1
Function<Integer, Integer> add1 = addN.apply(1);
```

Funciones de primera clase

- ▶ Además de como valor devuelto o argumento, las funciones también se pueden asignar a variables

```
Function<String, Integer> strToInt = Integer::parseInt;  
Function<Integer, Integer> add1 = n -> n + 1;
```

- ▶ Y pueden seguir siendo evaluadas como función

```
Integer value1 = strToInt.apply("3");  
Integer value2 = add1.apply(6);
```

Mónadas

- ▶ En el mundo funcional ideal la composición mediante funciones puras es sencilla
- ▶ El mundo real no es tan ideal
- ▶ Para salvar este obstáculo usamos las **mónadas**
 - ▶ Son una definición algebraica (teoría de categorías)
- ▶ Ejemplos:
 - ▶ Lidar con errores (`Try<T>`, `Result<T>`)
 - ▶ Lidar con valores nulos (`Optional<T>`)
 - ▶ Lidar con computación asíncrona (`CompletableFuture<T>`)

Curificación

- ▶ Transformar una función que toma múltiples argumentos en una secuencia de funciones, cada una con un único argumento
- ▶ Dada una función $f : (\mathbf{A} \times \mathbf{B}) \rightarrow \mathbf{C}$, la versión curificada es $h : \mathbf{A} \rightarrow (\mathbf{B} \rightarrow \mathbf{C})$
- ▶ ¿Dónde es útil?
 - ▶ En la composición de funciones
 - ▶ En las operaciones que esperan una función unaria
 - ▶ “Funciones especializadas” donde algunos argumentos se aplican
- ▶ En los lenguajes puramente funcionales está disponible por defecto, en Java no

Recursividad

- ▶ En la programación funcional no usamos la iteración
- ▶ Las operaciones complejas se resuelven usando recursividad (más declarativo)
- ▶ En los lenguajes puramente funcionales se optimiza usando la técnica **Tail-Call** o recursión de cola
- ▶ Esto significa que no tenemos que preocuparnos por la pila
- ▶ En Java, esta técnica no está disponible

Resumen

- ▶ Hemos visto las bases de la programación funcional
- ▶ Incluyendo sus principios fundamentales
- ▶ Lo que nos ofrece Java para elegir este paradigma en nuestros proyectos

Functional Programming in Java, Harnessing the Power of Java 8 Lambda Expressions. Venkat Subramaniam. The Pragmatic Programmers. 2014.

Java 8 in Action, Lambdas, streams, and functional-style programming. Raoul-Gabriel Urma, Mario Fusco, and Alan Mycroft. Manning. 2014

